

BedLog Raspberry Pi Installation Guide

Raspberry Pi 4 + PostgreSQL + Node.js Backend + Vite Frontend + 7-inch DSI Landscape Display + Chromium Kiosk + ESP32 API

Component	Configuration
Raspberry Pi	Raspberry Pi 4 Model B Rev 1.5
OS / Desktop	Raspberry Pi OS Trixie / labwc / Wayland
Project path	/home/pi/Documents/bedlog
PostgreSQL	PostgreSQL 17, database: bedlog, user: bedlog
Backend	Node.js service on port 4002
Frontend	Vite service on port 5173
Display	DSI-1, 720x1280 native, rotated to landscape
Pi LAN IP used here	192.168.68.121
ESP32 endpoint	http://192.168.68.121:4002/api/display/summary

1. Final System Architecture

```
Raspberry Pi boots
|
+-- PostgreSQL 17
|   |-- BedLog database
|   |
+-- bedlog-backend.service
|   |-- port 4002
|   |   +-- /health
|   |   +-- /api/backup
|   |   |-- /api/display/summary
|   |
+-- bedlog-vite.service
|   |-- port 5173
|   |
|-- labwc graphical session
    +-- rotate DSI-1 to landscape
    |-- Chromium kiosk -> http://localhost:5173

ESP32 -> http://192.168.68.121:4002/api/display/summary
```

2. Copy the BedLog Project from the Mac

Run this on the Mac. Do not copy node_modules or dist because the Mac packages are not compatible with Linux ARM.

```
rsync -av \
  --exclude node_modules \
  --exclude dist \
  "/Users/wealth/Downloads/Bedlog- with server/" \
  pi@192.168.68.121:/home/pi/Documents/bedlog/
```

Connect to the Pi and verify the project:

```
ssh pi@192.168.68.121

cd /home/pi/Documents/bedlog
ls -la
```

```
ls -la db-backend
```

The backend folder should contain package.json, schema.sql and server.cjs.

3. Install and Enable PostgreSQL

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
pg_lsclusters
```

The PostgreSQL 17 main cluster should report **online**. The wrapper unit may show active (exited); that is normal when the actual PostgreSQL cluster processes are running.

4. Create the BedLog Database

```
sudo -u postgres createuser bedlog
sudo -u postgres createdb bedlog -O bedlog
sudo -u postgres psql
```

At the **postgres=#** prompt:

```
ALTER USER bedlog WITH PASSWORD 'pi';
\q
```

Test the login:

```
PGPASSWORD=pi psql -h localhost -U bedlog -d bedlog
```

Exit with:

```
\q
```

For a production deployment, replace the example database password with a stronger password and update the backend service environment to match.

5. Import the Database Schema

```
PGPASSWORD=pi psql \
-h localhost \
-U bedlog \
-d bedlog \
-f /home/pi/Documents/bedlog/db-backend/schema.sql
```

Verify the tables:

```
PGPASSWORD=pi psql -h localhost -U bedlog -d bedlog
\dt
```

Expected tables: backup_log, beds, device_config, patients and wards. Exit with \q.

6. Install Node.js and npm

```
sudo apt install -y nodejs npm

node -v
npm -v
which node
which npm
```

The tested Pi used Node.js v20.19.2 and npm 9.2.0, with /usr/bin/node and /usr/bin/npm.

7. Remove Mac-only Frontend Dependencies

Edit the frontend package.json:

```
cd /home/pi/Documents/bedlog
```

```
nano package.json
```

Remove explicit Mac-only entries such as **@rollup/rollup-darwin-arm64** and **lightningcss-darwin-arm64** if present. Then rebuild the Linux dependency set:

```
rm -rf node_modules
rm -f package-lock.json
npm cache clean --force
npm install
npm install react@18.3.1 react-dom@18.3.1
npm list react react-dom
```

8. Install Backend Dependencies

```
cd /home/pi/Documents/bedlog/db-backend
rm -rf node_modules
npm install
```

9. Test the Frontend Manually

```
cd /home/pi/Documents/bedlog
npm run dev -- --host 0.0.0.0 --port 5173
```

In another terminal:

```
curl -I http://localhost:5173
```

Expected: HTTP/1.1 200 OK. Stop the manually started Vite server with Ctrl+C before systemd takes ownership.

10. Test the Backend Manually

```
cd /home/pi/Documents/bedlog/db-backend

PGHOST=localhost \
PGPORT=5432 \
PGDATABASE=bedlog \
PGUSER=bedlog \
PGPASSWORD=pi \
PORT=4002 \
npm start
```

In another terminal:

```
curl http://localhost:4002/health
curl http://localhost:4002/api/display/summary
```

Health should return `{"ok":true}`. The summary endpoint should return the current ward statistics once the frontend has backed up data.

11. Create the Backend Boot Service

First stop any manually started backend process occupying port 4002. Then create:

```
sudo nano /etc/systemd/system/bedlog-backend.service

[Unit]
Description=BedLog Database Backend
After=network-online.target postgresql.service
Wants=network-online.target
Requires=postgresql.service

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/Documents/bedlog/db-backend
Environment=HOME=/home/pi
Environment=PGHOST=localhost
Environment=PGPORT=5432
Environment=PGDATABASE=bedlog
```

```

Environment=PGUSER=bedlog
Environment=PGPASSWORD=pi
Environment=PORT=4002
ExecStart=/usr/bin/npm start
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable bedlog-backend.service
sudo systemctl start bedlog-backend.service
systemctl status bedlog-backend --no-pager

```

12. Create the Vite Boot Service

```

sudo nano /etc/systemd/system/bedlog-vite.service

[Unit]
Description=BedLog Vite Frontend
After=network-online.target bedlog-backend.service
Wants=network-online.target
Requires=bedlog-backend.service

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/Documents/bedlog
Environment=HOME=/home/pi
ExecStart=/usr/bin/npm run dev -- --host 0.0.0.0 --port 5173
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable bedlog-vite.service
sudo systemctl start bedlog-vite.service
systemctl status bedlog-vite --no-pager
curl -I http://localhost:5173

```

13. Rotate the 7-inch DSI Display to Landscape

The tested display is DSI-1 with a native 720x1280 mode. Check it with:

```
wlr-randr
```

Test a 90-degree landscape rotation:

```
wlr-randr --output DSI-1 --transform 90
```

If the physical orientation is reversed, use 270 instead:

```
wlr-randr --output DSI-1 --transform 270
```

To make the working transform run when the labwc desktop session starts:

```

mkdir -p /home/pi/.config/labwc
nano /home/pi/.config/labwc/autostart

```

Add the working rotation command, for example:

```
/usr/bin/wlr-randr --output DSI-1 --transform 90 &
```

After rotation, verify touchscreen alignment by touching all four corners. If touch does not follow the screen orientation, configure the touchscreen transform before final deployment.

14. Start Chromium as a Full-screen Kiosk

For a Wayland/labwc desktop, Chromium can be started from the user's labwc autostart after the graphical session is available. Edit:

```
nano /home/pi/.config/labwc/autostart
```

A practical BedLog autostart block is:

```
/usr/bin/wlr-randr --output DSI-1 --transform 90 &

sleep 10

/usr/bin/chromium \
  --ozone-platform=wayland \
  --kiosk \
  --start-fullscreen \
  --no-first-run \
  --disable-session-crashed-bubble \
  --disable-infobars \
  --noerrdialogs \
  --user-data-dir=/home/pi/.config/bedlog-chromium \
  http://localhost:5173 &
```

If 270 degrees is the correct landscape orientation, change the first line accordingly.

15. Avoid Duplicate Chromium Startup

If a separate bedlog-kiosk systemd service was created earlier, disable it when Chromium is being launched by labwc autostart:

```
sudo systemctl disable --now bedlog-kiosk.service
```

It is fine for PostgreSQL, the backend and Vite to remain systemd services while Chromium belongs to the graphical user session.

16. Enable Desktop Auto-login

Run:

```
sudo raspi-config
```

Configure the Pi to boot to the graphical desktop and automatically log in as user pi. This allows labwc to start and execute the display rotation and Chromium kiosk commands.

17. Verify Before Reboot

```
systemctl status postgresql --no-pager
systemctl status bedlog-backend --no-pager
systemctl status bedlog-vite --no-pager

curl http://localhost:4002/health
curl http://localhost:4002/api/display/summary
curl -I http://localhost:5173
```

The backend and Vite should be active (running), health should return ok:true, the summary endpoint should return ward data, and Vite should return HTTP 200.

18. Reboot and Confirm Automatic Startup

```
sudo reboot
```

Do not manually run npm start or npm run dev after reboot. The intended startup flow is:

```
POWER ON
|
```

```

+-- PostgreSQL
+-- bedlog-backend.service -> :4002
+-- bedlog-vite.service    -> :5173
`-- Desktop auto-login
    |-- labwc autostart
        +-- DSI-1 landscape
        |-- Chromium kiosk -> BedLog

```

After reboot, verify through SSH:

```

ssh pi@192.168.68.121

systemctl status bedlog-backend --no-pager
systemctl status bedlog-vite --no-pager

curl http://localhost:4002/health
curl http://localhost:4002/api/display/summary
curl -I http://localhost:5173

```

19. ESP32 API Configuration

The ESP32 must use the Raspberry Pi LAN address, not localhost:

```

const char* apiUrl =
    "http://192.168.68.121:4002/api/display/summary";

```

The endpoint returns JSON containing ward, admit, discharge, lama, deceased, abscond, total_beds, occupied and available.

20. Production Checklist

Check	Required result
PostgreSQL cluster	online
bedlog-backend.service	active (running)
bedlog-vite.service	active (running)
GET /health	{"ok":true}
GET /api/display/summary	current ward statistics
DSI-1	landscape
Chromium	full-screen kiosk
ESP32	can reach Pi port 4002
Router	reserve/static lease for 192.168.68.121 recommended

Important: 192.168.68.121 is the Pi's current LAN address. Reserve this address in the router/DHCP server, or update the ESP32 if the Pi's address changes.